

数据库工程作业

要求:

- 1. 完成一个小型的数据库信息管理系统（或部分功能），并填写工程作业报告；程序和报告请在规定时间之内上传。
- 2. 开发模式（B/S 或 C/S）、开发高级语言任选，后台数据库使用大型数据库管理系统（SQL Server、Oracle、MySQL 等），不要使用桌面数据库。
- 3. 报告中所列举的四种操作，每种操作举一个例子即可。
- 4. 作业成绩按照报告中的标准评分，程序只实现报告中涉及的部分即可。
- 5. 作业完成后，请将工程作业报告和程序打包提交给助教老师，并联系助教老师进行系统说明和演示，回答相关问题。

工程作业报告

1. 项目信息（10 分）

学号	2113615	姓名	张铭徐	专业	计算机科学与技术
项目名称	学生教师互动论坛+选课系统				
必备环境	Mysql+python+html				
系统主要功能简介（4 分）	论坛支持用户注册，自定义自己的用户类型，即老师或者同学，将账号密码以哈希加密的形式存储在数据库中。注册后用户可以通过自己注册的账号和密码进行登录。 学生和老师都可以在论坛中发帖和评论，但是为保护学生和老师的权限，我们在发帖时设定了权限，用户在发表帖子时可以指定老师或学生或全体可评论，只有有权限的用户才可以评论帖子。 支持搜索帖子功能，如果有必要，甚至可以在数据库中找到哪位用户在什么时间搜索了什么内容！此外，评论支持显示评论的用户名和评论时间。 对于课程模块，学生可以选课，教师可以开设课程，且仅有学生可以选课，教师可以开课，学生选课后，老师可以针对不同的课程对不同的学生进行打分，学生可以通过按钮查询对应的分数。 支持个人信息界面，对于老师，个人信息页面展示历史发过的帖子以及教授的课程，有便捷的打分按钮可以快速跳转到打分页面；对于学生，个人信息界面展示历史发过的帖子以及个人成绩，同样有便捷的按钮跳转到首页。				
系统主要页面截图（6 分）	论坛首页具有登录和注册两个选项： Welcome to our site! 本页面为数据库工程作业的先导页面，整体功能为学生老师互动论坛，论坛中包括发帖，评论，老师开课，老师给分，学生选课，学生查分等操作。 Powered By 2113615 张铭徐 Login Register				

可以选择登录或注册，假设我们已经有账号登录：

Show Posts

Search:

Enter search content

Search

帖子标题

123

123

Read more

现在是测试环节 测试的对象是学生权限

111123rt

Read more

什论坛

66666666666666666666666666666666, sb论坛赶紧倒闭

Read more

并行程序设计?

学不明白

Read more

Create a New Post

Go to Courses

个人信息

创建课程界面：

Create Course

Course Name:

Course Capacity:

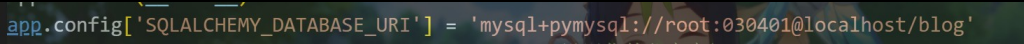
Create Course

学生选课界面：

Courses		
数据库系统	Teacher: lea Enrollment: 0	Enroll
Python程序设计	Teacher: lea Enrollment: 0	Enroll

	查分界面： My Grades Course: , Grade: 50.0 Course: , Grade: 50.0 Course: 数据库系统, Grade: None Course: Python程序设计, Grade: None
--	--

2. 系统配置（10 分）

说明		（2 分）请说明系统配置情况（后台数据库，高级语言）； （8 分）请使用连接串连接高级语言和数据库，并分析字符串的各个部分。		
配置 步骤 2 分	DBMS	1. mySQL		
		2. 无		
	高级 语言	1. Python		
		2. JavaScript		
连接串 分析 (6 分)	序 号	名称	功能说明	取值
	1	config	在 Python 代码中链接数据库以方面文件对数据库的修改	如下所示
	...			
	...			
连接串代码 (截屏) (2 分)				
备注	在 Python 中链接 localhost 的名为 blog 的数据库			

3. 数据库设计（14 分）

说明	（10 分）按照数据表的创建顺序，依次给出所涉及数据表的信息，其中参照字段以“（字段 1，字段 2，……，字段 n）”的形式给出，被参照字段以“表名（字段 1，字段 2，……，字段 n）”的形式给出； （4 分）一般 DBMS 都可以为数据库生成关系图，请将该图片截屏并粘贴到表格中。				
数据表 (10)	创建顺序	数据表名称	主键	参照属性	被参照表及属性
	1	User	UserID		
	2	Post	PostID	UserID	User 表中的 UserID
	3	Comment	CommentID	UserID	User 表中的 UserID
				PostID	Post 表中的 PostID
	4	course	CourseID	TeacherID	User 表中的 UserID
	5	Enrollment	EnrollmentID	StudentID	User 表中的 UserID

				CourseID	Course 表 中的 CourseID
	6	Search_history	SearchID	UserID	User 表中的 UserID
	7	Courseinfo	CourseID	StudentID	User 表中的 UserID
			StudentID	CourseID	Course 表 中的 CourseID
关系图 (4)	<pre>graph TD courseinfo --> course course --> enrollment search_history --> user user --> post user --> comment post --> comment</pre>				
备注					

4. 含有事务应用的删除操作（13 分）

说明	(1 分) 简要说明该操作所要完成的功能； (2 分) 该操作会涉及的表（必须含有两张或两张以上的关系表，同时以“表名”的形式给出） (1 分) 表连接涉及字段描述（描述方式为“表 1. 属性=表 2. 属性”） (1 分) 删除条件涉及的字段描述(以“表名. 属性=? ”形式给出) (4 分) 实现该操作的关键代码（高级语言、SQL），截图即可；（其中如果删除语句中不包含任何形式的事务应用将扣除 3 分） (4 分) 如何执行该操作，按所述方法能够正常演示程序则给分。
功能描述（1 分）	老师可以自由的创建课程，同时可以选择删除所创建的课程，在 enrollment 表中，存储了选课 ID，学生 ID，课程编号，成绩，以及已选学生人数，course 表中存储了课程的信息，课程名称，课程编号，学生 ID 和课程最大容量，我们删除课程时需要将 course 表中的数据删除，同时级联删除 enrollment 表中的对应行。

涉及的表 (2分)	enrollment 表和 course 表	
表连接涉及字段 (1分)	我们要删除 course 表的所有属性即课程名称，课程编号，学生 ID 和课程最大容量，同时根据 enrollment 表中的 courseID 删除对应列，两个表是通过 CourseID 链接的。	
删除条件字段描述 (1分)	字段	规则
	Enrollement.enrollment ID Enrollement.StudentID Enrollment.CourseID Enrollment.Grade Enrollment.Students Course.CourseID Course.CourseName Course.TeacherID Course.CourseCapacity	我们删除的原则为删除指定的课程编号(因为课程编号是课程表中的主键)
代码 (4分)	(截屏)  <pre>def withdraw_course(course_id): try: # 开启事务 db.session.begin() # 查询课程 course = Course.query.get(course_id) if course is None: raise Exception('Course not found.') # 删除课程 db.session.delete(course) # 删除关联的选课记录 enrollments = Enrollment.query.filter_by(CourseID=course_id).all() for enrollment in enrollments: db.session.delete(enrollment) # 提交事务 db.session.commit() return 'Course withdrawal successful.' except Exception as e: # 出现异常时回滚事务 db.session.rollback() return 'Course withdrawal failed: ' + str(e)</pre>	

```

START TRANSACTION;

-- 检查课程是否存在
SELECT * FROM course WHERE CourseID = 1 FOR UPDATE;

-- 删除课程
DELETE FROM course WHERE CourseID = 1;

COMMIT;

```

在网页后台可以有教师删除课程的按钮，如下所示：

Courses Taught:

- 数据库系统 - Enrolled: 2, Capacity: 10
Withdraw
- Python程序设计 - Enrolled: 2, Capacity: 10
Withdraw

Grade Students

Create Course

Show Posts

程序演示（4分）

目前数据库中有以下信息：

EnrollmentID	StudentID	CourseID	Grade	EnrolledStudents
9	1	2	85.6	0
11	5	2	72.3	0

CourseID	CourseName	TeacherID	CourseCapacity
1	数据库系统	2	10
2	Python程序设计	2	10

我对 CourseID 为 2 的课程进行退课操作：按下按钮后：他提示我：

Course withdrawal successful.

刷新页面后：

Courses Taught:

- 数据库系统 - Enrolled: 2, Capacity: 10
Withdraw

就没有了对应的课程，我们去数据库中看看对应的信息：

CourseID	CourseName	TeacherID	CourseCapacity
1	数据库系统	2	10

数据库中删除掉了对应的 Course 信息，我们再去看 Enrollment 表：

EnrollmentID	StudentID	CourseID	Grade	EnrolledStudents
8	1	1	(Null)	0
10	5	1	(Null)	0

刷新后就只有 CourseID 为 1 的课程了，剩下的内容全都被删除掉了。

接下来我们测试一下对应的 SQL 语句：我们执行：

START TRANSACTION;

SELECT * FROM course WHERE CourseID = 1 FOR UPDATE;

DELETE FROM course WHERE CourseID = 1;

COMMIT;

使用 SELECT FOR UPDATE 语句来锁定课程表中的对应记录，以确保其他事务无法修改该记录。接下来，执行删除选课记录的操作。最后，提交事务，完成删除操作。我们来测试一下这个代码的正确性，我们来到 Navicat 的 mysql 命令行界面：输入这行语句：

```
mysql> START TRANSACTION;

-- 检查课程是否存在
SELECT * FROM course WHERE CourseID = 1 FOR UPDATE;

-- 删除课程
DELETE FROM course WHERE CourseID = 1;

COMMIT;
Query OK, 0 rows affected (0.00 sec)

a
+-----+-----+-----+-----+
| CourseID | CourseName | TeacherID | CourseCapacity |
+-----+-----+-----+-----+
| 1 | 数据库系统 | 2 | 10 |
+-----+-----+-----+-----+
1 row in set (0.02 sec)

Query OK, 1 row affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql>
```

删除成功，我们来看一下涉及到删除操作的两个表格的内容：

EnrollmentID	StudentID	CourseID	Grade	EnrolledStudents
(N/A)	(N/A)	(N/A)	(N/A)	(N/A)

CourseID	CourseName	TeacherID	CourseCapacity
(N/A)	(N/A)	(N/A)	(N/A)

我们发现，对应的数据全被删除掉

备注

在上面的 SQL 中，我们使用了 START TRANSACTION 开始了一个事务，COMMIT 用于

	提交事务。通过在 SELECT 语句中使用 FOR UPDATE，我们锁定了课程表中对应记录，以确保其他事务无法修改该记录。最后，使用 DELETE 语句删除了课程。 此外，对于 Python 所负责的部分，我们使用了 try...expect 的操作：当执行删除操作时，我们将删除课程表中的记录以及关联的选课记录。在删除过程中，如果发生任何异常，将会回滚事务，确保数据库的一致性。 这些事务操作保证了在删除课程的过程中，要么全部操作成功，要么全部回滚，避免了数据不一致的情况。
--	--

5. 触发器控制下的添加操作（20 分）

说明	(1 分) 简要说明该操作所要完成的功能; (2 分) 简要说明该触发器所要完成的功能 (1 分) 该操作会涉及的表 (以 “表名” 的形式给出)。 (2 分) 该操作输入数据以及输入数据应该满足的条件, 如: 数值范围、是否为空; (6 分) 实现该操作的关键代码 (高级语言、SQL), 截图即可; (8 分) 如何执行该操作, 按所述方法能够正常演示程序则给分。	
功能描述 (1 分)	在这个部分中, 我实现了一个用户注册页面的触发器, 触发器的功能是判断用户所填写的用户名是否被使用过, 我们在这里的登录采取的逻辑是用户名+密码进行登录, 如果用户所设定的用户名和密码都完全相同, 我们是不知道如何区分这两个用户的, 会出现异常, 所以在这里我们要求用户的用户名是非空且唯一的, 触发器用于检测用户所填写的用户名在原来的数据库中是否存在, 如果存在则返回: Username already exists. 错误信息, 如果没问题, 那么就不会出现操作。 此外, 对于在 SQL 中插入操作, 我们也需要关注, 用户类型的问题, 用户类型也必须是 Student 或者是 Teacher, 插入的数据必须是这两者之一, 对于这一点, 我们所采取的操作是将 UserType 设置为枚举类型, 用户只能输入这两个之一, 否则会报错!	
触发器描述 (2 分)	触发器的具体细节如下: 1. 触发器名称: validate_username_trigger 2. 触发事件: BEFORE INSERT, 表示在执行插入操作之前触发触发器 3. 触发表: user, 即触发器所绑定的表名 4. 触发时间点: FOR EACH ROW, 表示对每一行插入操作都会触发触发器 5. 触发器主体: BEGIN...END 之间的代码块, 用于执行触发器的逻辑操作 6. 触发器逻辑: 使用 IF EXISTS 子查询来检查表中是否存在具有相同用户名的记录。如果存在, 则发出 SQLSTATE '45000' 的信号, 并将错误消息设置为 'Username already exists'。 这个触发器的目的是确保在插入新用户之前, 用户名是唯一的, 避免重复用户名的情况发生。	
涉及的表 (1 分)	User 表	
输入数据 (2 分)	字段	规则
	UserName	UserName 必须在数据库中仅出现过一次
	UserPassword	无
	UserType	输入数据必须是 Student or Teacher
插入操作 源码 (3 分)	(截屏) <pre>INSERT INTO user (UserName, UserPassword, UserType) VALUES ('Alice', 'hashed_password', 'Student');</pre>	
触发器源码 (3 分)	(截屏)	

	<pre>DELIMITER // CREATE TRIGGER validate_username_trigger BEFORE INSERT ON user FOR EACH ROW BEGIN IF EXISTS(SELECT 1 FROM users WHERE username = NEW.username) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Username already exists'; END IF; END // DELIMITER ;</pre>
程序演示 (4分)	← → ↻ ⓘ 127.0.0.1:5000/register User registered successfully! <pre>mysql> INSERT INTO user (UserName, UserPassword, UserType) VALUES ('Alice', 'hashed_password', 'Student'); Query OK, 1 row affected (0.01 sec)</pre>
程序演示 (4分)	OperationalError <pre>sqlalchemy.exc.OperationalError: pymysql.err.OperationalError (1644, 'Username already exists') [SQL: INSERT INTO user (UserName, UserPassword, UserType) VALUES (%(UserName)s, %(UserPassword)s, %(UserType)s)] (parameters: ('123', 'UserPassword', 'student')) Background on this error at: https://sqlalche.me/e/20/e3q8</pre> <pre>mysql> INSERT INTO user (UserName, UserPassword, UserType) VALUES ('Alice', 'hashed_password', 'Student'); 1644 - Username already exists</pre>
备注	针对于触发器，我们知道，触发器可以应用于数据一致性的维护，或者进行数据验证和约束，我们额外创建了一个针对 Update 和 Insert 的触发器，用于检测教师录入的成绩是否符合要求，其触发器代码如下所示；涉及到的表为 Enrollment 表： <pre>CREATE TRIGGER check_grade_range_insert_trigger BEFORE INSERT ON Enrollment FOR EACH ROW BEGIN IF NEW.Grade < 0 OR NEW.Grade > 100 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '成绩范围异常'; END IF; END; CREATE TRIGGER check_grade_range_update_trigger BEFORE UPDATE ON Enrollment FOR EACH ROW BEGIN IF NEW.Grade < 0 OR NEW.Grade > 100 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '成绩范围异常'; END IF; END;</pre> 针对这个功能，我们尝试修改成绩：

EnrollmentID	StudentID	CourseID	Grade	EnrolledStudents
1	1	500	-453430	0



我们可以看到，提示成绩范围异常，只能输入 0-100 之间的数据；此外，对于插入操作，也是同理：如下所示：

```
mysql> INSERT INTO Enrollment (StudentID, CourseID, Grade)
VALUES (3, 500, 5000);
1644 - 成绩范围异常
mysql>
```

如果我们的数据库更加复杂，例如我们针对每一个用户都建立一个表，表中存储用户发布的 PostID，选修的课程，那么我们在更新时就需要注意数据一致性的问题，在此时，除了外键约束之外，我们就可以考虑采用触发器进行数据一致性的校验和检测，以确保对某一张表进行更新后，所有表的数据都会被更新。

此外，对于 CourseInfo 表，我们也新建了若干触发器以保证数据一致性问题，在这里就不一一展示了。

6. 存储过程控制下的更新操作（18 分）

说明	（1 分）简要说明该操作所要完成的功能； （1 分）简要说明该存储过程所要完成的功能； （2 分）说明该操作涉及操作的表（必须包含两张或两张以上的关系表，以“表名形式”描述） （1 分）表连接涉及字段描述（描述方式为“表 1. 属性=表 2. 属性”） （2 分）该操作会修改字段（以“表名. 字段名”的形式给出），以及修改规则，如新数值的计算方法、在何种条件下予以修改等； （6 分）实现该操作的关键代码（高级语言、SQL），截图即可； （5 分）如何执行该操作，按所述方法能够正常演示程序则给分。	
功能描述 (1 分)	我们有若干张表涉及学生的成绩，例如 Enrollment 表，courseinfo 表，同时也有若干张表设计课程 ID，enrollment 表，courseinfo 表以及 Course 表，我们期望于在更新一张表时将所有的表的对应参数都更新好，事实上这个操作在外键约束的 Update 中，只需要将其设计为 CASCADE 即可，在这里我们考虑应用存储过程来进行更新操作，并且设定一定的约束条件，例如在这里，我们要求课程 ID 就算被修改，也只能是正整数，并且学生成绩一定是在 0-100 的区间之内。 值得注意的是，在这里，我们 courseinfo 仅仅存储某一门课程的全部学生的信息，包括学生姓名，学生 ID，学生成绩。	
存储过程 功能描述 (1 分)	我们新建了两个存储过程，分别是 UpdateStudentGrade 和 UpdateCourseID；对于第一个更新学生成绩的存储过程，我们需要传递两个参数，分别是 StudentID 和 Grade，调用该存储过程后，将自动的将 Courseinfo 表中所涉及到的学生成绩修改为对应的新成绩，同时更新选课 enrollment 表中的数据。 对于第二个更新课程 ID 的存储过程，我们同样传递两个参数，分别是旧课程 ID 和新课程 ID，调用该存储过程后，会将 Course，Enrollment 以及 CourseInfo 中的对应 CourseID 都进行修改。	
涉及的关系表(2分)	Enrollment, Course, Courseinfo	
表连接涉及字段(1)	CourseID	
更改字段 (2 分)	字段	规则
	Grade	我们首先在在 Courseinfo 中更改某一名学生的成绩，然后根据 StudentID，将 Enrollment 表中所对应的成绩也进行更改。
	CourseID	我们同样传递两个参数，分别是旧课程 ID 和新课程 ID，调用该存储过程后，会将 Course, Enrollment 以及 CourseInfo 中的对应 CourseID 都进行修改。
更新代码 (3 分)	<pre> UPDATE CourseInfo SET Grade = p_Grade WHERE CourseID IN (SELECT CourseID FROM Enrollment WHERE StudentID = p_StudentID) -- 执行更新操作 UPDATE Enrollment SET Grade = p_Grade WHERE CourseID IN (SELECT CourseID FROM CourseInfo WHERE StudentID = p_StudentID) AND StudentID=p_StudentID; -- 执行更新操作 UPDATE Course SET CourseID = p_NewCourseID WHERE CourseID = p_OldCourseID; -- 级联更新Enrollment表中的数据 UPDATE Enrollment SET CourseID = p_NewCourseID WHERE CourseID = p_OldCourseID; UPDATE CourseInfo SET CourseID = p_CourseID WHERE CourseID = p_OldCourseID; </pre>	

创建存储过程源码 (3 分)	<pre> -- 创建存储过程：更新学生成绩并级联更新Enrollment表 CREATE PROCEDURE UpdateStudentGrade(IN p_StudentID INT, IN p_Grade INT) BEGIN -- 约束条件：成绩必须在0-100之间 IF p_Grade < 0 OR p_Grade > 100 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Grade must be between 0 and 100.'; ELSE UPDATE CourseInfo SET Grade = p_Grade WHERE CourseID IN (SELECT CourseID FROM Enrollment WHERE StudentID = p_StudentID) -- 执行更新操作 UPDATE Enrollment SET Grade = p_Grade WHERE CourseID IN (SELECT CourseID FROM CourseInfo WHERE StudentID = p_StudentID) AND StudentID=p_StudentID; END IF; END // DELIMITER ; DELIMITER // -- 创建存储过程：更新课程ID并级联更新Course和Enrollment表 CREATE PROCEDURE UpdateCourseID(IN p_OldCourseID INT, IN p_NewCourseID INT) BEGIN -- 约束条件：新的课程ID必须是正整数 IF p_NewCourseID <= 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'New Course ID must be a positive integer.'; ELSE -- 执行更新操作 UPDATE Course SET CourseID = p_NewCourseID WHERE CourseID = p_OldCourseID; -- 级联更新Enrollment表中的数据 UPDATE Enrollment SET CourseID = p_NewCourseID WHERE CourseID = p_OldCourseID; UPDATE CourseInfo SET CourseID = p_NewCourseID WHERE CourseID = p_OldCourseID; END IF; END // DELIMITER ; </pre>
存储过程执行源码 (1 分)	<pre> mysql> call updatestudentgrade(9,85); Query OK, 1 row affected (0.01 sec) </pre>

mysql> call updatecourseid(1,8);
Query OK, 0 rows affected (0.01 sec)

在这里我们演示两个在前端和数据库两端进行操作：

Grade Students

Select Course: 测试更新

Student ID	
9	10
10	20
11	30
12	40
13	50
14	60
15	70
16	80
17	90
18	100
20	66

Submit

设置分数后，我们查看后台：

CourseID	StudentID	StudentName	Grade
8	9	Student1	10
8	10	Student2	20
8	11	Student3	30
8	12	Student4	40
8	13	Student5	50
8	14	Student6	60
8	15	Student7	70
8	16	Student8	80
8	17	Student9	90
8	20	123	85

EnrollmentID	StudentID	CourseID	Grade	EnrolledStudents
1	9	8	10	0
2	10	8	20	0
3	11	8	30	0
4	12	8	40	0
5	13	8	50	0
6	14	8	60	0
7	15	8	70	0
8	16	8	80	0
9	17	8	90	0
10	18	8	100	0
22	20	8	66	0
23	20	3	(Null)	0

程序演示
(2分)

我们发现无论是 enrollment 表还是 Courseinfo 表的 grade 都被更改了。
我们还可以展示一下前端显示成绩：

Course Details	
Course Name	测试更新
Course ID	8
Teacher ID	19
Teacher Name	Teacher1
Enrollment Count	11

Enrolled Students:

Student ID	Student Name	Grade
9	Student1	10.0
10	Student2	20.0
11	Student3	30.0
12	Student4	40.0
13	Student5	50.0
14	Student6	60.0
15	Student7	70.0
16	Student8	80.0
17	Student9	90.0
18	Student10	100.0
20	123	66.0

接下来是数据库操作更改成绩：假设我们要更改学号为 9 的学生的成绩，原来学生的成绩为 10 分，我要更改为 55 分：

```
mysql> call updatestudentgrade(9,55);
Query OK, 1 row affected (0.00 sec)
```

CourseID	StudentID	StudentName	Grade
8	9	Student1	55

EnrollmentID	StudentID	CourseID	Grade	EnrolledStudents
1	9	8	55	0

我们发现，成功进行了修改！
接下来我们展示一下修改课程编号，现在是 8 号，我们希望更改为 1 号：

```
mysql> call updatecourseid(8,1);
Query OK, 0 rows affected (0.01 sec)
```

EnrollmentID	StudentID	CourseID	Grade	EnrolledStudents
1	9	1	55	0
2	10	1	20	0
3	11	1	30	0
4	12	1	40	0
5	13	1	50	0
6	14	1	60	0
7	15	1	70	0
8	16	1	80	0
9	17	1	90	0
10	18	1	100	0
22	20	1	66	0

CourseID	CourseName	TeacherID	CourseCapacity
1	测试更新	19	300
3	第二课程	19	20

成功进行了修改！

程序演示 (2 分)	由于我们有要求, 课程分数必须是 0-100 之间的数, 并且课程编号必须是正整数, 我们来测试一下: <pre>mysql> call updatestudentgrade(9,-59); 1644 - Grade must be between 0 and 100.</pre> <pre>mysql> call updatecourseid(1,-1); 1644 - New Course ID must be a positive integer.</pre> 发现, 确实与我们所期望弹出的提示相同, 故我们成功进行了存储过程的更新操作!
备注	一般而言, 课程编号是不会被修改的, 但是在极其特殊的情况下可能会对其进行修改, 我们在前端网页不会对教师授予权限进行更改, 只有在数据库的后台中才可以看到修改的页面!

7. 含有视图的查询操作（15 分）

说明	（1 分）简要说明该操作所要完成的功能； （1 分）简要说明建立的该视图的功能； （2 分）简要说明该操作涉及的关系数据表（以“表名”的形式给出） （1 分）简要说明表连接涉及的字段（以“表 1. 属性=表 2. 属性”） （6 分）实现该操作的关键代码（高级语言、SQL），截图即可； （4 分）如何执行该操作，按所述方法能够正常演示程序则给分。
操作功能描述（1 分）	查询学生的选课信息，包括学生学号、学生姓名、课程名称和成绩。
视图功能描述（1 分）	创建一个视图，命名为 StudentCourseView，用于展示学生的选课信息，包括所有学生学号、学生姓名、课程名称和成绩。
涉及的关系表（2 分）	User 表：存储学生信息，包含学生学号和学生姓名。 Enrollment 表：存储选课信息，包含学生学号、课程 ID 和成绩。 Course 表：存储课程信息，包含课程 ID 和课程名称。
表连接字段（1 分）	表连接涉及的字段： User.UserID = Enrollment.StudentID：通过学生学号关联 User 表和 Enrollment 表。 Course.CourseID = Enrollment.CourseID：通过课程 ID 关联 Course 表和 Enrollment 表。
创建视图代码	（截屏）

码
(3
分)

```
1 SELECT
2     `user`.UserID AS StudentID,
3     `user`.UserName AS StudentName,
4     course.CourseName AS CourseName,
5     enrollment.Grade AS Grade
6 FROM
7     (
8         (
9             `user`
10            join
11            enrollment
12            ON
13            (
14                (
15                    `user`.UserID = enrollment.StudentID
16                )
17            )
18        )
19        join
20        course
21        ON
22        (
23            (
24                course.CourseID = enrollment.CourseID
25            )
26        )
27    )
```

查
询
代
码
(3
分)

```
@app.route('/admin')
def admin():
    # 执行查询操作，获取view2的数据
    query = text('SELECT * FROM view2')
    result = db.session.execute(query)
    view2_data = result.fetchall()
    # 渲染admin.html视图，并将查询结果传递给模板
    return render_template('admin.html', view2_data=view2_data)
```

程
序
演
示
(4
分)

```
1644 - New Course ID must be a positive integer.
mysql> select * from view2;
+-----+-----+-----+-----+
| StudentID | StudentName | CourseName | Grade |
+-----+-----+-----+-----+
| 9 | Student1 | 测试更新 | 60 |
| 10 | Student2 | 测试更新 | 60 |
| 11 | Student3 | 测试更新 | 60 |
| 12 | Student4 | 测试更新 | 60 |
| 13 | Student5 | 测试更新 | 60 |
| 14 | Student6 | 测试更新 | 60 |
| 15 | Student7 | 测试更新 | 60 |
| 16 | Student8 | 测试更新 | 60 |
| 17 | Student9 | 测试更新 | 60 |
| 18 | Student10 | 测试更新 | 60 |
| 20 | 123 | 测试更新 | 60 |
| 20 | 123 | 第二课程 | NULL |
+-----+-----+-----+-----+
12 rows in set (0.06 sec)
```

前端:

Admin Page

Student ID	Student Name	Course Name	Grade
9	Student1	测试更新	60.0
10	Student2	测试更新	60.0
11	Student3	测试更新	60.0
12	Student4	测试更新	60.0
13	Student5	测试更新	60.0
14	Student6	测试更新	60.0
15	Student7	测试更新	60.0
16	Student8	测试更新	60.0
17	Student9	测试更新	60.0
18	Student10	测试更新	60.0
20	123	测试更新	60.0
20	123	第二课程	None

我们额外做了一个视图，包括 comment 表，和 user 表，在这个视图中，显示所有用户的用户名，用户 ID，评论的内容，评论 ID，还有评论时间，帖子 ID 等内容，表表之间通过 comment.userid=user.userid 来链接，在这里我们给出创建视图的 SQL：

```
CREATE ALGORITHM = UNDEFINED SQL SECURITY DEFINER VIEW `view2` AS select
`user`.`UserID` AS `StudentID`,`user`.`UserName` AS
`StudentName`,`course`.`CourseName` AS `CourseName`,`enrollment`.`Grade` AS
`Grade` from ((`user` join `enrollment` on((`user`.`UserID` =
`enrollment`.`StudentID`))) join `course` on((`course`.`CourseID` =
`enrollment`.`CourseID`)));
```

路由函数的代码如下：可以实现查询登录用户的评论内容

```
@app.route('/user_comments')
def user_comments():
    user_id = session.get('user_id')
    if user_id is None:
        return redirect(url_for('login'))
    query = text('SELECT * FROM user_comment_view WHERE UserID = :user_id')
    result = db.session.execute(query, {'user_id': user_id})
    comments = result.fetchall()
    return render_template('user_comments.html', comments=comments)
```

查询结果如下所示

User Comments

Comment ID: 2

Comment Content: 全微分个魔法师
Comment Date: 2023-05-21
Post ID: 2

Comment ID: 3

Comment Content: 分额外分而测我
Comment Date: 2023-05-21
Post ID: 2

Comment ID: 4

Comment Content: 期望微分物权法而过去
Comment Date: 2023-05-21
Post ID: 2

备注